

Asistente de Emergencias Viales

Verificación completa, cambios aplicados y resultados · septiembre 2026

Este documento resume la auditoría del asistente de voz (LiveKit Agents + RAG sobre el manual de primeros auxilios), los problemas encontrados, cada cambio que se hizo en el agente y en el frontend, cómo se verificó y qué queda pendiente. Todos los cambios están en el working tree, **sin commit ni push**.

Resultados en una tabla

Indicador	Antes	Después
Guion de la Expo (verificaciones automáticas)	18 / 23	23 / 23
«No respira»	«Perdí el acceso al manual», sin RCP	Indica compresiones en el centro del pecho
Hemorragia grave	«Eso no está en mi manual»	«Apretá fuerte sobre la herida con una prenda limpia»
«¿Le saco el casco?»	Ignoraba la pregunta	«No le quites el casco; mantené el cuello inmóvil»
RCP en curso («¿ahora qué?»)	«¿Ves que el pecho se mueve?»	«Seguí comprimiendo sin parar...»
Búsquedas en el manual que fallaban	5 de 6 (error 429 de Google)	0 en los temas críticos (caché precalculado)
Latencia por turno (texto: LLM + búsqueda)	1,3 a 7,8 s	~0,5 a 0,8 s en turnos críticos; ~1 a 2 s si el modelo registra datos
Aviso «Ya estás geolocalizado...»	Dependía del modelo; a veces faltaba	Lo agrega el sistema, una vez por llamada
Tests unitarios	12 pasan, 9 se saltean	48 pasan, 9 se saltean

Las latencias varían entre corridas por la variación propia de Groq; se observaron picos aislados de hasta ~3 s. Se midieron por texto, sin STT ni TTS.

1. Diagnóstico inicial

Se corrieron los tests unitarios, los ocho escenarios de ensayo.py, la batería test_guion_expo.py y mediciones aisladas de cada fase contra los servicios reales.

Problemas críticos (seguridad)

- **El embedding de Google (plan gratuito) fallaba en ~70 % de las llamadas** con 429 «Resource exhausted», aun espaciadas 3 s: es falta de capacidad del lado de Google, no un problema de ritmo. Resultado: el agente no indicaba RCP a una persona que no respira y decía «no está en mi manual» ante una hemorragia.
- **El límite de 7 s del reranker bloqueaba el turno.** La key de Cohere es Trial (10 llamadas/min) y rag/ratelimit.py esperaba el cupo dentro de una búsqueda con tope de 4 s, así que una segunda búsqueda terminaba en timeout.
- **El paro no se detectaba después de «inconsciente»:** el detector solo miraba la primera señal de la llamada.

- **Los avisos de riesgo desaparecían en el turno siguiente:** se agregaban a `turn_ctx`, que en LiveKit vale solo para ese turno.

Problemas de calidad

- Ignoraba preguntas directas («¿le saco el casco?», «¿muevo el auto?») para seguir con su orden de triage.
- Tras «ya hice las compresiones» pedía revisar si el pecho se movía, en vez de seguir con la RCP.
- Ante «no veo si reacciona» preguntaba «¿está despierto?».
- Jerga prohibida («vía aérea») y voseo neutro («verifica»); dos preguntas por turno; preguntaba lo que ya le habían dicho.

Dónde se iba el tiempo

Fase	Medido	Causa
LLM por llamada	~2,5 s (~1900 tokens)	gpt-oss-120b razonaba en modo «medium»: el plugin solo fija <code>reasoning_effort</code> para GPT-5
LLM con <code>reasoning_effort=«low»</code>	~0,55 a 0,68 s	—
Llamadas al LLM por turno	2, una detrás de otra	<code>registrar_datos_escena</code> y <code>derivar_a_emergencias</code> casi en cada turno
Embedding (Google)	0,5 a 1,4 s, o error	429 y reintentos
Búsqueda en Supabase	430 a 510 ms	226 ms de ida y vuelta (us-west-2) para un corpus de solo 173 fragmentos
Rerank (Cohere)	0,4 a 1,2 s, o hasta 7 s	Key Trial y throttle bloqueante
Conexión a la base	~1,9 s	Se hacía dentro del primer turno que buscaba en el manual

2. Cambios para velocidad

Cambio	Detalle	Archivo
reasoning_effort=«low»	Para modelos gpt-oss en Groq. Configurable con LLM_REASONING_EFFORT.	agent.py (create_llm)
Índice en memoria	Al arrancar se cargan los 173 fragmentos con sus embeddings. Coseno en numpy (<1 ms) y BM25 en memoria. La base queda como camino de respaldo.	rag/index.py, rag/store.py
Caché de consultas precalculado	Embedding + ranking rerankeado de 12 consultas canónicas. Los temas críticos se resuelven en 0 ms, sin Google ni Cohere.	rag/cache_consultas.json, ingestion/cache_consultas.py
Protocolo adjuntado antes del LLM	Si se detecta un tema (RCP, hemorragia, casco...), el protocolo se agrega al mensaje y el modelo responde en una sola pasada, sin llamar a buscar_protocolo.	agent.py (contexto_del_turno)
Sin herramientas en turnos críticos	Lo esencial ya queda registrado por el sistema; se le indica al modelo que responda directo (ahorra ~1 s).	agent.py, triage.py
Sin tool derivar_a_emergencias	La derivación al 911 es automática y determinística: no hace falta otra vuelta al LLM.	triage.py
Rerank sin espera	intentar_turno(): si no hay cupo se usa el orden del coseno al instante.	rag/ratelimit.py, rag/rerank.py
Prewarm del proceso	Base, índice y caché se cargan antes de la primera llamada (AgentServer setup_fnc).	agent.py
Fin de turno	max_delay de 3,0 a 2,0 s. En el chat, el sleep(0.2) se reemplazó por esperar la interrupción.	agent.py

3. Cambios para confiabilidad del manual (RAG)

Orden de búsqueda nuevo, del más rápido y confiable al menos:

- **Resultado precalculado** (caché): 0 ms, sin red.
- **Embedding de Google** (3 intentos rápidos) + coseno en memoria + rerank de Cohere solo si hay cupo.
- **Búsqueda por palabras clave (BM25)** en memoria si el embedding falla. Tiene un piso de cobertura para no devolver fragmentos sin relación.
- **Texto de respaldo verificado** (protocolos.py) para los temas críticos si todo lo anterior falla: la maniobra nunca queda sin decirse.

Temas críticos (protocolos.py)

Diez temas, cada uno con palabras de detección, consulta canónica al manual y un resumen redactado **solo** a partir de fragmentos recuperados del manual: RCP, inconsciente que respira, hemorragia, fuego, atrapado, columna, casco, quemadura, fractura y seguridad de la escena.

Convulsiones no es un tema: el manual no las trata y la búsqueda devolvía el fragmento de RCP con relevancia 0,27. Inyectarlo habría sido peligroso; ante una convulsión el agente avisa que la ayuda va en camino y no inventa maniobras.

4. Cambios para la calidad de las respuestas

- **Detección por turno y por categoría** (paro, atrapamiento, inconsciencia, hemorragia, fuego, convulsión). El paro se avisa aunque antes se haya dicho «inconsciente», y el sistema marca solo «no respira» o «atrapado».
- **Los avisos persisten**: se agregan al mensaje del usuario (en voz, en el chat y en los scripts de prueba por igual).
- **Resumen verificado + fragmentos**: con solo los fragmentos, el modelo indicaba la posición lateral a un inconsciente que respira; el manual dice que en un accidente de tránsito no se lo mueve salvo que vomite.
- **Preguntas primero**: se detecta si la persona preguntó algo y se le exige responderlo en ese turno. Además, la tool registrar_datos_escena deja de empujar a «datos que faltan» cuando hay una pregunta o un protocolo recién adjuntado.
- **RCP segura**: nunca se adjunta el protocolo de compresiones sin haber confirmado que no respira. Durante la RCP, cada turno recuerda «seguí sin parar», y «volvió a respirar» pasa al protocolo del inconsciente que respira.
- **Sin acceso al herido**: si la persona no ve o no llega, no se le pide que revise nada.
- **Brevedad**: una indicación de hasta dieciocho palabras por turno.
- **Corrección del texto antes de hablar** (habla.py, en Assistant.llm_node): «vía aérea» se cambia por «el paso del aire», «fractura» por «hueso roto», «100 a 120 por minuto» por «dos veces por segundo» y «112» por «nueve once», y se corrige el voseo («verifica» por «verificá»). No toca preguntas en tercera persona como «¿Respira?».
- **Prompt reescrito**: prioridades explícitas por turno, una sola pregunta, no repetir datos ya dichos, no narrar lo que registra, respuesta correcta a temas ajenos (VTV, seguros).

5. Aviso «Ya estás geolocalizado y la ayuda va en camino»

Para la demo, la frase es determinística. Cuando el sistema deriva al 911 (heridos confirmados o riesgo de vida), se agrega al principio de la primera respuesta, **una sola vez por llamada**. Si el modelo la repite después, se quita. Si no hay heridos ni riesgo, no se dice. En el panel de debug la derivación sigue apareciendo como derivar_a_emergencias (automática).

Caso	Primera respuesta después de derivar
Inconsciente	«Ya estás geolocalizado y la ayuda va en camino. Fijate si se le mueve el pecho. ¿Respira?»
Hemorragia	«Ya estás geolocalizado y la ayuda va en camino. Poné una gasa o prenda limpia sobre la herida...»
Humo y combustible, sin heridos	«Ya estás geolocalizado y la ayuda va en camino. Alejate del vehículo...»
Choque leve sin heridos	No se menciona

6. Panel de debug del frontend

Como ahora la mayoría de los protocolos entran por el sistema (prefetch) y no por la tool, el agente registra prefetch_protocolo con los mismos datos que buscar_protocolo: fragmentos con sección, página y score, latencias, contexto exacto enviado al LLM y un campo nuevo **«Origen»** (cache, semantico, lexico, db o respaldo). En TestTab.tsx esas tarjetas muestran el detalle completo con la etiqueta «adjuntado por el sistema · tema». El contador de búsquedas suma las dos vías.

7. Archivos modificados y nuevos

Archivo	Estado	Qué cambió
agent.py	Modificado	reasoning_effort, prewarm, contexto_del_turno, llm_node con correcciones y aviso 911, buscar_protocolo con respaldo, debug
triage.py	Modificado	Detección por categoría, derivación automática, avisos nuevos, sin tool derivar_a_emergencias
prompts.py	Modificado	Prompt reescrito
rag/retriever.py	Modificado	Caché, índice en memoria, respaldo léxico, métricas por búsqueda sin estado compartido
rag/store.py	Modificado	cargar_todo() para el índice en memoria
rag/embeddings.py	Modificado	Reintentos configurables
rag/rerank.py, rag/ratelimit.py	Modificado	Cupo sin espera en vivo; espera solo offline
ensayo.py, test_guion_expo.py	Modificado	Usan el mismo contexto del turno que el agente; escenarios casco y pregunta directa
requirements.txt	Modificado	livekit-agents 1.6.5 a 1.6.8 (versión probada); numpy explícito
AGENTS.md, docs/guion_demo_expo.md	Modificado	Arquitectura y comandos nuevos
protocolos.py	Nuevo	Temas críticos, detección y textos de respaldo
habla.py	Nuevo	Correcciones de habla y aviso 911
rag/index.py	Nuevo	Índice en memoria (coseno + BM25)
rag/cache_consultas.json	Nuevo	Caché precalculado (12 consultas)
ingestion/cache_consultas.py	Nuevo	Genera el caché
tests/test_turno.py	Nuevo	36 tests de la lógica determinística
asistente-frontend/app/components/TestTab.tsx	Modificado	Detalle de prefetch_protocolo y campo Origen

8. Cómo se verificó

- **Tests unitarios:** 48 pasan (9 se saltean porque requieren el corpus reconstruido).
- **Guion de la Expo** contra Groq real: 23/23 verificaciones.
- **Escenarios de ensayo.py** (10, incluidos casco y pregunta directa) revisados turno por turno.
- **Caché regenerado** con el índice en memoria: 12/12 consultas rerankeadas.
- **Payload de debug:** se simuló un turno y se revisó el JSON que recibe el frontend.
- **Frontend:** npm ci, TypeScript sin errores, ESLint limpio en TestTab.tsx y next build correcto.

No verificado: una llamada real por voz. En audio falta probar la detección en on_user_turn_completed, las correcciones antes de Cartesia y el prewarm dentro de LiveKit Cloud.

9. Cómo probarlo

Por texto, en la consola (en PowerShell):

```
$env:LOG_LEVEL="WARNING"
./venv/Scripts/python.exe ensayo.py --interactivo
./venv/Scripts/python.exe ensayo.py --escenario casco
./venv/Scripts/python.exe test_guion_expo.py
```

Con voz y panel de debug:

```
# terminal 1, asistente_de_emergencias
./venv/Scripts/python.exe agent.py dev

# terminal 2, asistente-frontend (requiere .env.local con LIVEKIT_*)
npm run dev
```

Al arrancar el agente, el log tiene que mostrar «índice en memoria: 173 fragmentos» y «caché de consultas: 12 embeddings, 12 resultados listos».

Después de cada reingesta de PDFs:

```
python -m ingestion.cache_consultas --mostrar
```

Despliegue:

```
lk agent deploy          # agente
# y desplegar también el frontend, para ver las tarjetas de prefetch
```

10. Pendientes y riesgos

- **Probar una llamada real por voz** antes de desplegar (ver la sección 9).
- **Facturación en Google AI Studio** (opcional, cuesta centavos): eliminaría los 429 de raíz en consultas fuera del caché. Sin ella, esas consultas caen a la búsqueda por palabras clave.
- **Versiones:** el Dockerfile usa Python 3.11 y el venv local, 3.13.
- **Métricas:** tres scripts de metrics/ importan search_similarity desde agent.py, que ya no existía antes de estos cambios, y no corren.
- **README:** dice que los embeddings son de Cohere; son de Gemini.
- **Git:** con core.autocrlf, algunos diffs se ven completos por los finales de línea; `git diff --ignore-cr-at-eol` muestra solo los cambios reales. Nada está commiteado.